

MATERIA: Análisis y Metodología

Segunda entrega:

“Del análisis al Producto Web”

Profesor: Sebastián Stenico

Alumnos:

Lassalle Nora

Hernández Andrés

27/06/2026

"SmartGastro"



SmartGastro

Segunda Entrega — Del Análisis al Producto Web

Campo	Detalle
Materia	Análisis y Metodología de Sistemas
Docente	Juan Sebastián Stenico
Instituto	Universidad Da Vinci
Año	2026
Integrantes	Lassalle Nora / Hernández Andrés
Fecha	Junio 2026

➤ Links de entrega

Repositorio GitHub: <https://github.com/git-nora/smartgastro>

Tablero Kanban (Trello): <https://trello.com/invite/b/6a3f4f5c8921ad2a2d4ce37b/ATTI3f9a97dd74f136fad8d479464d524c5f022BF299/smartgastro>

Descripción del Proyecto

SmartGastro es un MVP (Producto Mínimo Viable) desarrollado en Python con Flask que permite a dueños de foodtrucks gestionar su negocio desde el navegador. El sistema resuelve un problema real: los foodtrucks pierden mercadería cuando llueve y se quedan sin stock en pleno evento. SmartGastro alerta cuando hay pronóstico de lluvia y permite controlar el inventario en tiempo real.

➤ Tecnologías utilizadas

Tecnología	Uso
Python 3.14.6	Lenguaje principal
Flask 3.0	Framework web
SQLAlchemy	ORM - Base de datos relacional
SQLite	Base de datos (persistencia)
bcrypt	Encriptación de contraseñas
JWT	Autenticación por token

Jinja2	Motor de plantillas HTML
OpenWeatherMap API	Datos climáticos en tiempo real
Git / GitHub	Control de versiones

➤ Estructura del repositorio

Archivo / Carpeta	Descripción
smartgastro.py	Motor lógico en consola (Primera Entrega)
app.py	Servidor Flask y rutas de vistas web
auth.py	Autenticación JWT (registro y login)
routes.py	Endpoints API REST con CRUD completo
models.py	Modelos de base de datos con SQLAlchemy
extensions.py	Extensiones Flask (db, bcrypt)
clima.py	Integración con API OpenWeatherMap
templates/	Vistas HTML renderizadas con Jinja
static/	Archivos CSS y JavaScript
.env.example	Variables de entorno de ejemplo
requirements.txt	Dependencias del proyecto

Gestión Ágil — Tablero Kanban

El tablero Kanban fue gestionado en Trello durante todo el desarrollo, con tarjetas redactadas como Historias de Usuario siguiendo el criterio de valor de negocio (no tareas técnicas). A continuación se muestran las capturas que evidencian el avance del proyecto commit a commit.

➤ Historias de Usuario

ID	Historia de Usuario	Responsable
HU-01	Como administrador, quiero registrarme con usuario y contraseña encriptada para acceder al sistema de forma segura	Nora
HU-02	Como administrador, quiero iniciar sesión con mis credenciales para obtener un token de acceso al panel	Nora
HU-03	Como desarrollador, quiero que las rutas de gestión estén protegidas para que nadie acceda sin estar autenticado	Nora
HU-04	Como dueño, quiero que los datos se guarden en una base de datos para no perderlos al cerrar el sistema	Nora

HU-05	Como dueño, quiero ver una alerta en el dashboard cuando hay pronóstico de lluvia para reducir la producción	Nora
HU-06	Como cajero, quiero registrar la venta de un producto para descontar el stock automáticamente	Andrés
HU-07	Como administrador, quiero agregar un nuevo producto desde el navegador sin recargar la página	Andrés
HU-08	Como dueño, quiero ver el listado de proveedores para saber a quién contactar cuando necesito reponer mercadería	Andrés
HU-09	Como operador, quiero registrar la locación del foodtruck para planificar los eventos de la semana	Andrés
HU-10	Como dueño, quiero ver un dashboard principal con el resumen del negocio para tomar decisiones rápidas	Andrés

➤ Evolución del tablero — Capturas por commit

Commit	Descripción	Captura
00	Estado inicial — todas las tareas en Por Hacer	00_estado_inicial.JPG
01	feat: estructura base del proyecto Flask	01_commit1_inicio.JPG
02	feat: modelos de base de datos con SQLAlchemy	02_commit2_modelos.JPG
03	feat: autenticación - registro y login con JWT	03_commit3_auth.JPG
04	feat: CRUD productos con protección de rutas	04_commit4_productos.JPG
05	feat: CRUD ventas y proveedores	05_commit5_ventas.JPG
06	feat: integración API clima OpenWeatherMap	06_commit6_clima.JPG
07	feat: frontend dashboard con Jinja y alerta clima	07_commit7_frontend.JPG
08	feat: alta de producto asíncrona con fetch()	08_commit8_fetch.JPG
09	fix: validaciones y manejo de errores con rollback	09_commit9_validaciones.JPG
10	docs: actualización README segunda entrega	10_commit10_readme.JPG

Integración API Climática — OpenWeatherMap

El sistema consume la API pública de OpenWeatherMap para obtener el clima actual de Buenos Aires al cargar el Dashboard. Si hay pronóstico de lluvia, se muestra una alerta visual recomendando reducir la producción del día. La API key se guarda en el archivo `.env` y nunca en el código fuente.

➤ Endpoint consultado

Campo	Valor
Método	GET
URL	https://api.openweathermap.org/data/2.5/weather?q=Buenos+Aires&appid;={API_KEY}&units;=metric
Endpoint interno	GET /api/clima (protegido con JWT)
Respuesta exitosa	200 OK con datos de temperatura, humedad, descripción y flag hay_lluvia

➤ Pasos para reproducir la prueba en Postman

1. Levantar el servidor con python app.py
2. Hacer POST a http://127.0.0.1:5000/api/auth/login con body:

```
1 {
2   "username": "Nora",
3   "password": "Nora919"
4 }
```

3. Copiar el token JWT de la respuesta
4. Hacer GET a http://127.0.0.1:5000/api/clima con Authorization: Bearer {token}

➤ Respuesta exitosa de la API

```
1 {
2   "alerta": null,
3   "ciudad": "Buenos Aires",
4   "descripcion": "Nubes",
5   "error": false,
6   "hay_lluvia": false,
7   "humedad": 66,
8   "icono": "04d",
9   "temperatura": 15.1
10 }
```

Ver imágenes adjuntas:

- Obteniendo token desde Postman
- Token
- Realizando consulta con token
- Respuesta a funcionamiento app de clima